Nome:Kevin N.Meireles

Questao VIII

Recuperação da Questão 8 de A1 – ACD - 2026/2 [Peso original: 1.5]

MergeSort de três vias (*MergeSort3Way*)

Nesta variante do *MergeSort*, o vetor é dividido em três partes, que são ordenadas recursivamente e, em seguida, combinadas por uma fusão de três vias.

(a) [0,5] Implemente, em pseudocódigo, o método `MergeSort3Way(A, p, r)`, que divide o subvetor `A[p..r]` em três partes, ordena cada uma recursivamente e chama o método auxiliar `Merge3Way`.

`MergeSort3Way(A, p, r)`

PSEUDO-CÓDIGO

```
MergeSort3Way(A, p, r)
   // Ordena o subvetor A[p..r] (inclusivo) por MergeSort de 3 vias
   // Caso base: 0 ou 1 elemento já está ordenado
   se p >= r então
      retorne

   // Divide A[p..r] em três partes (o mais equilibradas possível)
   // tamanho total = r - p + 1
   n   = r - p + 1
   t1  = p + ⌊n/3⌋ - 1          // fim da 1ª parte
   t2  = p + ⌊2n/3⌋ - 1         // fim da 2ª parte
   // 1ª parte: A[p .. t1]
   // 2ª parte: A[t1+1 .. t2]
   // 3ª parte: A[t2+1 .. r]

   // Ordena recursivamente cada uma das três partes
   MergeSort3Way(A, p,     t1)
   MergeSort3Way(A, t1+1,  t2)
   MergeSort3Way(A, t2+1,  r)
```

```
    // Combina as três partes ordenadas
    Merge3Way(A, p, t1, t2, r)
```

MÉTODO AUXILIAR

```
Merge3Way(A, p, t1, t2, r)
    // Fusão de três vias: A[p..t1], A[t1+1..t2], A[t2+1..r] já ordenados
    n1 = t1 - p + 1
    n2 = t2 - t1
    n3 = r  - t2

    // Copia as três partes para vetores auxiliares L, M, R
    para i = 0 até n1-1 faça L[i] = A[p + i]
    para i = 0 até n2-1 faça M[i] = A[t1 + 1 + i]
    para i = 0 até n3-1 faça R[i] = A[t2 + 1 + i]

    i = 0; j = 0; k = 0; q = p
    enquanto i < n1 e j < n2 e k < n3 faça
        se L[i] <= M[j] e L[i] <= R[k] então
            A[q] = L[i]; i = i + 1
        senão se M[j] <= L[i] e M[j] <= R[k] então
            A[q] = M[j]; j = j + 1
        senão
            A[q] = R[k]; k = k + 1
        q = q + 1
    // (aqui seguiriam as fusões de dois restantes e do último restante,
    //  análogas ao Merge de 2 vias, até esgotar as três listas)
```

(b) [0,7] Implemente o método `Merge3Way(A, p, q1, q2, r)`, que combina três subvetores previamente ordenados em um único subvetor ordenado.
`Merge3Way(A, p, q1, q2, r)`

Os índices q1 e q2, calculados no método `MergeSort3Way(A, p, r),` *delimitam as três partes (vias) de ordenação:*
- A[p...q1]

- A[q1+1…q2]
- A[q2+1…r]
No método `MergeSort3Way(A, p, r)`, deverá ser calculado os pontos de divisão e tratar o(s) caso(s)-base - o caso base, neste caso das 3 vias, trata de subvetores de tamanhos um (1) e dois (2), ao contrário do método original em que o caso base é um subvetor de um só elemento. Já no método `Merge3Way(A, p, q1, q2, r)` deve-se implementar a fusão a partir das três vias (no MergeSort original, a fusão é feita a partir de 2 vias), preservando a ordem crescente.

```
public class MergeSort3Way {

    /**
     * Ordena o subvetor A[p..r] usando MergeSort de 3 vias.
     * Casos-base: subvetores de tamanho 1 (já ordenado) e tamanho 2
     * (ordenado com uma comparação simples).
     */
    public static void mergeSort3Way(int[] A, int p, int r) {
        // Caso base: vetor vazio ou de um único elemento -> já ordenado
        if (p >= r) {
            return;
        }

        int n = r - p + 1;

        // Caso base: subvetor de 2 elementos -> ordena diretamente
        if (n == 2) {
            if (A[p] > A[p + 1]) {
                int tmp = A[p];
                A[p] = A[p + 1];
                A[p + 1] = tmp;
            }
            return;
        }

        // Calcula os dois pontos de divisão, tentando dividir em 3 partes iguais
        // n = tamanho; tamanho de cada parte base = n/3
        int terco = n / 3;

        int q1 = p + terco - 1;          // fim da 1ª via: A[p..q1]
        int q2 = q1 + terco;             // fim da 2ª via: A[q1+1..q2]
```

```
    // Garante consistência dos índices
    if (q1 < p) q1 = p;
    if (q2 < q1 + 1) q2 = q1 + 1;
    if (q2 >= r) q2 = r - 1;

    // Ordena recursivamente cada uma das três vias
    mergeSort3Way(A, p, q1);
    mergeSort3Way(A, q1 + 1, q2);
    mergeSort3Way(A, q2 + 1, r);

    // Funde as três vias ordenadas
    merge3Way(A, p, q1, q2, r);
}

/**
 * Funde três subvetores já ordenados em um único subvetor ordenado.
 *   Via 1: A[p..q1]
 *   Via 2: A[q1+1..q2]
 *   Via 3: A[q2+1..r]
 */
public static void merge3Way(int[] A, int p, int q1, int q2, int r) {
    int n1 = q1 - p + 1;          // tamanho da via 1
    int n2 = q2 - q1;             // tamanho da via 2
    int n3 = r - q2;              // tamanho da via 3

    // Vetores auxiliares para cada via
    int[] L = new int[n1];
    int[] M = new int[n2];
    int[] R = new int[n3];

    for (int i = 0; i < n1; i++) L[i] = A[p + i];
    for (int i = 0; i < n2; i++) M[i] = A[q1 + 1 + i];
    for (int i = 0; i < n3; i++) R[i] = A[q2 + 1 + i];

    int i = 0, j = 0, k = 0;      // índices em L, M, R
    int t = p;                    // índice de escrita em A

    // Enquanto houver elementos nas três vias, escolhe o menor dos três
    while (i < n1 && j < n2 && k < n3) {
        if (L[i] <= M[j] && L[i] <= R[k]) {
            A[t++] = L[i++];
        } else if (M[j] <= L[i] && M[j] <= R[k]) {
            A[t++] = M[j++];
        } else {
            A[t++] = R[k++];
        }
    }
```

```
        // Esgotou a via 3: funde apenas L e M (2 vias)
        while (i < n1 && j < n2) {
            if (L[i] <= M[j]) A[t++] = L[i++];
            else              A[t++] = M[j++];
        }
        // Esgotou a via 2: funde apenas L e R (2 vias)
        while (i < n1 && k < n3) {
            if (L[i] <= R[k]) A[t++] = L[i++];
            else              A[t++] = R[k++];
        }
        // Esgotou a via 1: funde apenas M e R (2 vias)
        while (j < n2 && k < n3) {
            if (M[j] <= R[k]) A[t++] = M[j++];
            else              A[t++] = R[k++];
        }

        // Restou apenas uma via com elementos -> copia o restante
        while (i < n1) A[t++] = L[i++];
        while (j < n2) A[t++] = M[j++];
        while (k < n3) A[t++] = R[k++];
    }

    // -------- Teste rápido --------
    public static void main(String[] args) {
        int[] A = {38, 27, 43, 3, 9, 82, 10, 1, 55, 7, 99, 12, 5, 66, 21};
        mergeSort3Way(A, 0, A.length - 1);
        for (int x : A) System.out.print(x + " ");
        System.out.println();
        // Saída esperada: 1 3 5 7 9 10 12 21 27 38 43 55 66 82 99
    }
}
```

(c) [0,3] Analise a ordem de crescimento do tempo de execução do algoritmo implementado, comparando com o MergeSort original e justificando sua resposta.

O MergeSort 3-Way tem recorrência T(n) = 3T(n/3) + Θ(n). Pela árvore de recursão (cada nível custa cn e há $\log_3 n$ níveis) ou pelo Teorema Mestre (Caso 2, pois n^{$\log_3 3$} = n = f(n)), obtém-se T(n) = Θ(n $\log_3 n$) = Θ(n log n). Logo, sua ordem de crescimento é a mesma do MergeSort original de 2 vias (Θ(n log n)), diferindo apenas na constante: o 3-way tem árvore mais baixa ($\log_3 n$ níveis), mas fusão mais cara (compara até 3 elementos e usa 3 vetores auxiliares). O espaço também é Θ(n) em ambos.

\